

Documentation de gestion de projet

— Vite & Gourmand

Auteur : Dylan Quinzin

Formation : Titre Professionnel Développeur Web et Web Mobile (RNCP 37674) — Studi

Cadre : Évaluation en Cours de Formation (ECF)

Dépôt GitHub : <https://github.com/dquinzin1-jpg/vite-et-gourmand>

Outil de gestion de projet : <https://github.com/users/dquinzin1-jpg/projects/1>

Application déployée : <https://dp-vite-et-gourmand.alwaysdata.net>

1. Contexte du projet

Vite & Gourmand est un projet fictif réalisé dans le cadre de mon ECF pour le Titre Professionnel Développeur Web et Web Mobile. Il s'agit de développer, de A à Z, une application web pour un traiteur bordelais (Julie et José), permettant de présenter des menus, de gérer des commandes en ligne et d'administrer l'activité selon trois rôles (client, employé, administrateur).

L'objectif de l'évaluation est de démontrer la maîtrise de l'ensemble des compétences du référentiel, aussi bien sur la partie front-end (maquettage, interfaces statiques et dynamiques) que back-end (base de données relationnelle et non relationnelle, composants métier, sécurité, déploiement).

2. Objectifs du projet

- **Objectif pédagogique :** mettre en pratique, sur un cas réaliste, l'ensemble des activités-types du référentiel TP DWWM (développement front-end sécurisé et back-end sécurisé).
- **Objectif fonctionnel :** livrer une application complète et déployée en ligne, conforme au cahier des charges fourni dans le sujet (espaces public, client, employé, administrateur).
- **Objectif technique :** respecter les contraintes imposées par le sujet, notamment l'utilisation conjointe d'une base de données relationnelle (MariaDB) et d'une base de données non relationnelle (MongoDB), ainsi que le déploiement sur un serveur public.

3. Méthodologie de gestion de projet

3.1 Outil de suivi : tableau Kanban GitHub Projects

J'ai utilisé **GitHub Projects** comme outil de gestion, sous la forme d'un tableau Kanban intitulé « *Vite & Gourmand – Suivi de développement* ». Cet outil est directement intégré à mon dépôt de code, ce qui évite la dispersion entre plusieurs plateformes. Les tâches ont été organisées en trois colonnes selon leur état d'avancement — **Faire, En cours et Fait** — afin de visualiser la progression et de prioriser le travail au jour le jour. Le tableau distingue ainsi les tâches terminées, celles en cours de réalisation (documentation) et un reste à faire incluant des axes d'amélioration identifiés.

3.2 Versionnage avec Git et GitHub

J'ai appliqué les bonnes pratiques Git demandées par le sujet, avec une structure de branches claire :

- Une branche principale `main`, qui contient les versions stables et testées du projet.
- Une branche de développement `dev`, sur laquelle est intégré le travail courant.
- Des **branches de fonctionnalité** issues de `dev` pour les développements structurants, notamment :
 - `feature/structure` — mise en place de l'architecture du projet (dossiers config, includes, pages, assets) ;
 - `feature/database` — conception et création de la base de données relationnelle ;
 - `feature/config` — configuration de l'environnement (connexion BDD, fichier `.env`).

Le principe appliqué était : créer une branche de fonctionnalité à partir de `dev`, développer et tester la fonctionnalité, puis fusionner (*merge*) la branche vers `dev`. Une fois `dev` stable et testée, fusion vers `main`.

En fin de projet, par souci de rapidité sur des évolutions plus ciblées (filtres dynamiques de la page menus, intégration MongoDB), certaines fonctionnalités ont été développées directement sur la branche `dev` puis intégrées à `main`.

3.3 Convention de commits

J'ai adopté une **convention de commits sémantiques** pour garder un historique lisible, avec un préfixe indiquant la nature du changement :

- `feat:` pour une nouvelle fonctionnalité ;
- `security:` pour une correction de sécurité ;
- `chore:` pour une tâche de maintenance (configuration, outillage) ;
- `docs:` pour la documentation.

Exemples de commits réels du projet :

- `feat: ajout transaction SQL et insertion suivi_commande`

- docs: ajout des 3 diagrammes UML
- chore: enrichissement gitignore
- security: retrait du fichier .env du tracking Git
- feat: integration MongoDB Atlas pour stats admin
- feat: filtres dynamiques sans rechargement sur la page menus (JS pur)
- feat: frais de livraison hors Bordeaux (5 EUR + 0,59 EUR/km)
- feat: notifications mail simulees (bienvenue + confirmation commande)
- fix: liens de menu-detail utilisant BASE_URL

4. Planning et déroulement réel

Le projet a été réalisé sur une période courte et intensive, du 16 au 21 mai 2026. Le dépôt Git a été initialisé le **samedi 16 mai 2026 à 17h26**. Le déroulement réel, reconstitué à partir de l'historique des commits, a été le suivant :

Jour	Travail réalisé
Samedi 16 mai	Initialisation du projet, création de la base de données relationnelle (fichier SQL), mise en place de la structure du projet et de l'ensemble des pages PHP, authentification complète (bcrypt) et espaces multi-rôles sécurisés, et premier déploiement sur AlwaysData avec une <code>BASE_URL</code> dynamique.
Dimanche 17 mai	Refonte design premium (Bootstrap 5) et captures d'écran, rédaction de la charte graphique et du manuel d'utilisation (HTML + PDF), réalisation des maquettes fonctionnelles, ajout du formulaire d'avis client.
Lundi 18 mai	Première version des filtres dynamiques de la page menus, ajout des maquettes mobiles, création du fichier <code>seed-data.sql</code> (données de démonstration), mise en place de la transaction SQL atomique de commande avec insertion du suivi, et ajout des 3 diagrammes UML (cas d'utilisation, classes, séquence).
Mardi 19 mai	Travaux de sécurité : enrichissement du <code>.gitignore</code> , retrait du fichier .env du suivi Git, puis intégration de MongoDB Atlas pour les statistiques de l'espace administrateur.
Mercredi 20 mai	Finalisation des filtres dynamiques de la page menus sans rechargement (JavaScript pur), puis ajout des frais de livraison hors Bordeaux (5 € + 0,59 €/km), des notifications par e-mail (bienvenue à l'inscription et confirmation de commande) et correction des liens de la page détail (utilisation de <code>BASE_URL</code>). Synchronisation finale de la branche <code>main</code> .
20-21 mai	Rédaction de la documentation technique et de la documentation de gestion de projet, finalisation et dépôt.

On notera que le **déploiement en production a été réalisé dès le premier jour**, ce qui m'a permis de tester en continu le comportement de l'application sur le serveur cible tout au long du développement, et non au dernier moment.

5. Difficultés rencontrées et solutions apportées

Plusieurs difficultés techniques ont jalonné le projet. Les présenter ici fait partie de la démarche : elles montrent ma capacité à diagnostiquer un problème et à prendre une décision.

5.1 Refonte de la base de données

En cours de projet, j'ai dû retravailler la structure de la base de données pour la rendre plus cohérente, en adoptant notamment une convention de nommage homogène pour les clés (`xxx_id`). Cette refonte a fiabilisé les relations entre les tables et facilité l'écriture des requêtes.

5.2 Intégration de MongoDB en production

Le sujet impose l'usage d'une base de données non relationnelle. J'ai mis en place une **architecture hybride** : MariaDB reste la source de vérité transactionnelle (commandes, comptes, menus), et MongoDB Atlas sert de base analytique pour alimenter le graphique de statistiques de l'espace administrateur (via un *pipeline* d'agrégation `$match` / `$group` / `$sort`).

L'extension PHP MongoDB fonctionne correctement **en local** (XAMPP). Sur le serveur de production AlwaysData, je l'ai compilée via PECL : elle fonctionne en ligne de commande (CLI), mais **pas en mode Web** (configuration Apache cgi-fcgi). Après diagnostic, j'ai pris une **décision pragmatique** : conserver le fonctionnement MongoDB en environnement local, documenter précisément la limite rencontrée et l'inscrire comme **axe d'amélioration** (par exemple via un service externe ou une alternative d'hébergement supportant l'extension en mode Web).

5.3 Configuration PHP sur le serveur (php.ini / CGI)

Le déploiement a nécessité d'intervenir sur la configuration PHP du serveur (fichier `php.ini` personnalisé) pour activer certaines extensions. Le mode d'exécution PHP du serveur (CGI/FastCGI) a généré des comportements différents de l'environnement local, ce qui m'a appris l'importance de ne jamais considérer qu'un code testé en local fonctionnera à l'identique en production.

5.4 Fuite d'identifiants dans l'historique Git

J'ai constaté que le fichier `.env` (contenant les identifiants de connexion à la base) avait été versionné par erreur depuis le début du projet. J'ai traité l'incident de manière rigoureuse : retrait du fichier du suivi Git (`git rm --cached`), enrichissement du `.gitignore`, et **régénération de l'ensemble des mots de passe concernés**

(MariaDB et MongoDB). Cet incident m'a sensibilisé à la gestion des secrets dans un projet versionné.

5.5 Encodage des caractères (UTF-8)

Certaines données affichaient des caractères accentués incorrects. J'ai corrigé l'encodage des données concernées (noms de menus, plats, allergènes) en local via des requêtes SQL de mise à jour. Les données de production étaient déjà correctes, ce qui n'a pas nécessité de correction supplémentaire en ligne.

5.6 Corruption de la base de données locale (moteur Aria)

En fin de projet, le serveur MySQL local (MariaDB via XAMPP) a refusé de démarrer après un arrêt non propre de la machine. Plutôt que de réinstaller à l'aveugle, j'ai lu le fichier de log d'erreur, qui pointait clairement la cause : une corruption des journaux du moteur de stockage Aria (`Aria recovery failed` , `Could not open mysql.plugin table`). Après avoir constaté que mes données applicatives (stockées en InnoDB) étaient intactes, j'ai réalisé une sauvegarde du dossier `data` , puis supprimé les fichiers de journaux Aria corrompus (`aria_log*`) que MariaDB régénère automatiquement au démarrage. Le serveur est reparti sans aucune perte de données. Cet épisode m'a appris à diagnostiquer une panne à partir des logs plutôt que de procéder par tâtonnement.

6. Apprentissages et bilan

Ce projet m'a permis de progresser sur plusieurs aspects essentiels du métier de développeur :

- **Organisation et versionnage** : structurer un projet avec des branches Git claires et une convention de commits lisible facilite réellement le suivi et la reprise du travail.
- **Sécurité** : au-delà de la théorie, j'ai mis en pratique le hachage des mots de passe, les requêtes préparées, la protection des sessions, et j'ai vécu concrètement l'importance de la gestion des secrets.
- **Différence local / production** : plusieurs difficultés (MongoDB, php.ini, encodage) m'ont montré qu'un déploiement n'est jamais une simple copie de fichiers et qu'il faut anticiper les spécificités du serveur cible.
- **Prise de décision** : face à un blocage technique (MongoDB en mode Web), savoir diagnostiquer puis trancher de façon pragmatique, tout en documentant la limite, fait partie intégrante du travail de développeur.